

Open ▾ +

*Practical12a.c

/mnt/c/WINDOWS/system32/os/practical

S

```
1
2 #include <stdio.h>
3 #include <pthread.h>
4 #include <semaphore.h>
5
6 #define SIZE 5
7 #define ITEMS 10
8
9 int buffer[SIZE];
10 int in = 0, out = 0;
11
12 sem_t empty, full;
13 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
14
15 void *producer(void *arg) {
16     for (int i = 1; i <= ITEMS; i++) {
17         sem_wait(&empty);
18         pthread_mutex_lock(&mutex);
19
20         buffer[in] = i;
21         printf("Produced: %d\n", i);
22         in = (in + 1) % SIZE;
23
24         pthread_mutex_unlock(&mutex);
25         sem_post(&full);
26     }
27
28     return NULL;
29 }
30
31 void *consumer(void *arg) {
32     for (int i = 1; i <= ITEMS; i++) {
33         sem_wait(&full);
34         pthread_mutex_lock(&mutex);
35
36         int item = buffer[out];
37         printf("Consumed: %d\n", item);
38         out = (out + 1) % SIZE;
39
40         pthread_mutex_unlock(&mutex);
41         sem_post(&empty);
42     }
43
44     return NULL;
45 }
46
47 int main() {
48     pthread_t p, c;
49
50     sem_init(&empty, 0, SIZE);
51     sem_init(&full, 0, 0);
52
53     pthread_create(&p, NULL, producer, NULL);
54     pthread_create(&c, NULL, consumer, NULL);
55 }
```

```

27
28     return NULL;
29 }
30
31 void *consumer(void *arg) {
32     for (int i = 1; i <= ITEMS; i++) {
33         sem_wait(&full);
34         pthread_mutex_lock(&mutex);
35
36         int item = buffer[out];
37         printf("Consumed: %d\n", item);
38         out = (out + 1) % SIZE;
39
40         pthread_mutex_unlock(&mutex);
41         sem_post(&empty);
42     }
43
44     return NULL;
45 }
46
47 int main() {
48     pthread_t p, c;
49
50     sem_init(&empty, 0, SIZE);
51     sem_init(&full, 0, 0);
52
53     pthread_create(&p, NULL, producer, NULL);
54     pthread_create(&c, NULL, consumer, NULL);
55
56     pthread_join(p, NULL);
57     pthread_join(c, NULL);
58
59     sem_destroy(&empty);
60     sem_destroy(&full);
61     pthread_mutex_destroy(&mutex);
62
63     printf("Producer-Consumer completed\n");
64     return 0;
65 }

```

```

ganesh@GANESH:/mnt/c/WINDOWS/system32/os/practical$ gcc Practical12a.c -o Practical12a
ganesh@GANESH:/mnt/c/WINDOWS/system32/os/practical$ ./Practical12a
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Produced: 6
Produced: 7
Produced: 8
Produced: 9
Produced: 10
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
Consumed: 10
Producer-Consumer completed

```

Open

▼

+

Practical12b.c

/mnt/c/WINDOWS/system32/os/practical

```
1
2 #include <stdio.h>
3 #include <pthread.h>
4
5 pthread_mutex_t r1 = PTHREAD_MUTEX_INITIALIZER;
6 pthread_mutex_t r2 = PTHREAD_MUTEX_INITIALIZER;
7
8 void *thread1(void *arg) {
9     pthread_mutex_lock(&r1);
10    printf("Thread 1 acquired Resource 1\n");
11
12    pthread_mutex_lock(&r2);
13    printf("Thread 1 acquired Resource 2\n");
14
15    pthread_mutex_unlock(&r2);
16    pthread_mutex_unlock(&r1);
17
18    return NULL;
19 }
20
21 void *thread2(void *arg) {
22    pthread_mutex_lock(&r1);
23    printf("Thread 2 acquired Resource 1\n");
24
25    pthread_mutex_lock(&r2);
26    printf("Thread 2 acquired Resource 2\n");
27
28    pthread_mutex_unlock(&r2);
29    pthread_mutex_unlock(&r1);
30
31    return NULL;
32 }
33
34 int main() {
35    pthread_t t1, t2;
36
37    pthread_create(&t1, NULL, thread1, NULL);
38    pthread_create(&t2, NULL, thread2, NULL);
39
40    pthread_join(t1, NULL);
41    pthread_join(t2, NULL);
42
43    printf("No deadlock occurred\n");
44
45    pthread_mutex_destroy(&r1);
46    pthread_mutex_destroy(&r2);
47
48    return 0;
49 }
```

ganesh@GANESH:/mnt/c/WINDOWS/system32/os/practical\$ gcc Practical12b.c -o Practical12b
ganesh@GANESH:/mnt/c/WINDOWS/system32/os/practical\$./Practical12b
Thread 2 acquired Resource 1
Thread 2 acquired Resource 2
Thread 1 acquired Resource 1
Thread 1 acquired Resource 2
No deadlock occurred
ganesh@GANESH:/mnt/c/WINDOWS/system32/os/practical\$